UCAM

Tema 1

Introducción a Android

Aplicaciones para Dispositivos Móviles

Daniel Valero Carreras

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1. ¿Qué es Android hoy?
2. Herramientas necesarias
3. Kotlin en Android
4. Estructura de un proyecto Android
5. Componentes de una app de Android Compose
6. Primer contacto con Jetpack Compose

1. ¿Qué es Android hoy?

Definición

Imagen 1
Logos de Android



Esta foto de Autor desconocido está bajo licencia CC BY-SA-NC

1. ¿Qué es Android hoy?
Historia

- 2003 → Nace Android Inc. (Andy Rubin y equipo)
- 2005 → Google adquiere Android.
- 2007 → Presentación oficial y creación de la Open Handset Alliance.
- 2008 → Primer dispositivo Android: HTC Dream (T-Mobile G1)
- 2011 → Android supera a iOS en cuota de mercado global.

Imagen 2
Andy Rubin



Nota. Andy Rubin con un móvil Android

1. ¿Qué es Android hoy?
Historia

- 2014 → Google anuncia Material Design
- 2017 → Kotlin se convierte en lenguaje oficial de Android
- 2020 → Nace Jetpack Compose (UI declarativa).
- 2023+ → Compose recomendado como estándar para nuevas apps.

Imagen 3
Jetpack Compose

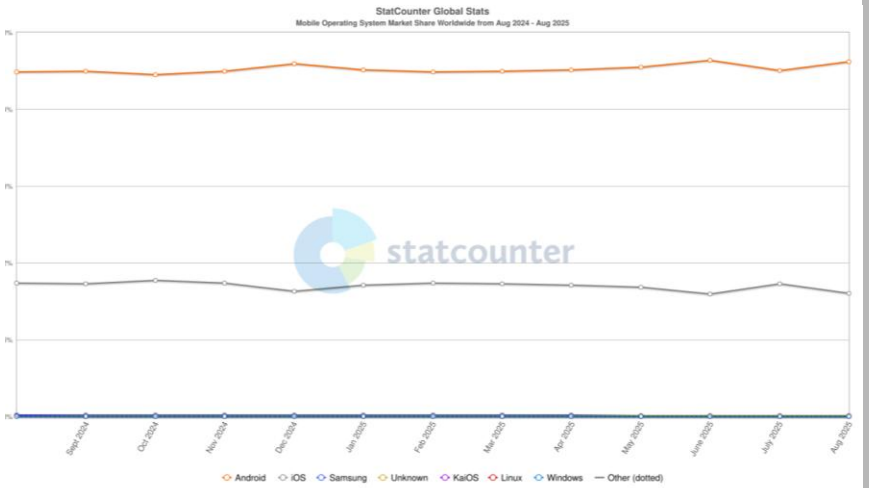


Nota. Logo Jetpack Compose

1. ¿Qué es Android hoy?
Historia

- Más de 3.000 millones de dispositivos activos
- Lenguaje oficial: **Kotlin**
- UI: **Jetpack Compose** (reemplazo progresivo de XML)
- Arquitectura: **Android Jetpack**
- Filosofía: desarrollo declarativo, rápido y mantenible

Imagen 4
Estadísticas uso Android



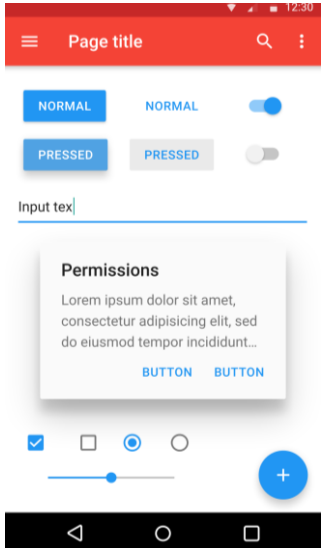
Nota. Gráfica uso sistemas operativos móviles, Statcounter.com

1. ¿Qué es Android hoy?
Ecosistema actual

- **Android Jetpack**
- **Kotlin**
- **Jetpack Compose**
  - Nueva forma declarativa de construir interfaces.
  - Reemplazo progresivo de XML.
- **Material Design 3**
  - Guías de diseño de Google para apps consistentes y accesibles.

Imagen 5

Pantalla realizada con Material Design



Nota. Una vista de los elementos que conforman el material design, Wikipedia

1. ¿Qué es Android hoy?

Mercado laboral y aplicaciones reales

- Android = +70% cuota del mercado de smartphones
- Sueldo medio en España:
  - 28000-46000 euros
- Conocimientos clave demandados:
  - Kotlin
  - Jetpack Compose
  - Arquitectura MVVM / Clean Architecture
  - Consumo de APIs REST

Imagen 6
Trabajadores



Nota. Imagen de trabajadores

2. Herramientas necesarias

Definición

Imagen 7
Pantalla con herramientas



Nota. Pantalla de la que salen iconos de herramientas

2. Herramientas necesarias
Android Studio

- ✓ ¿Qué es Android Studio?
  - Es el SDK (Software Development Kit) para Android, que incluye todas las características necesarias para programar aplicaciones Android así como un IDE (Integrated Development Environment) para desarrollar el código.
- ✓ ¿Qué es un SDK?
  - Un kit de desarrollo de software (SDK) es un conjunto de herramientas que ofrece generalmente el fabricante de una plataforma de hardware, un sistema operativo (SO) o un lenguaje de programación.
- ✓ ¿Qué es un IDE?
  - Un entorno de desarrollo integrado (IDE) es un sistema de software para el diseño de aplicaciones que combina herramientas comunes para desarrolladores en una sola interfaz de usuario gráfica (GUI).

2. Herramientas necesarias

Android Studio

- ✓ Tener instalado JAVA:



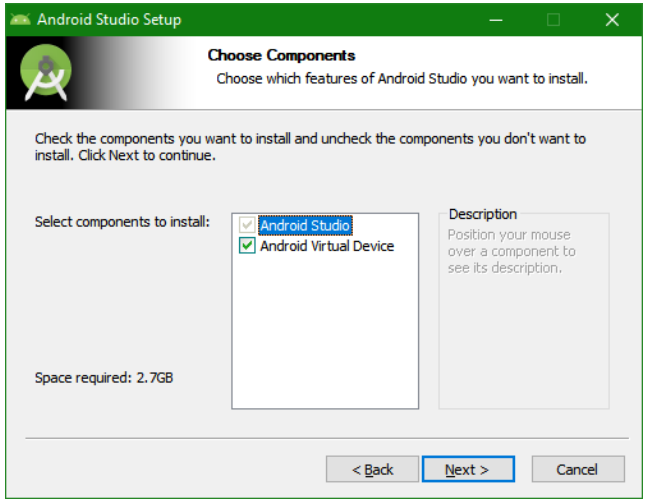
- ✓ Descargar el SDK de Android:



2. Herramientas necesarias
Android Studio

- Instalar SDK
  - Ir a la URL: developer.android
  - Darle a descargar (“Download Android Studio”) y acepta los “Términos y Condiciones” que aparecen en PopUp.
  - Ejecutar el instalador.
  - Darle a ‘Siguiente’ en todas las pantallas dejando los valores por defecto.
    - NOTA: Asegurarse que esté marcado “Android Virtual Device”.

Imagen 7
Instalador Android Studio

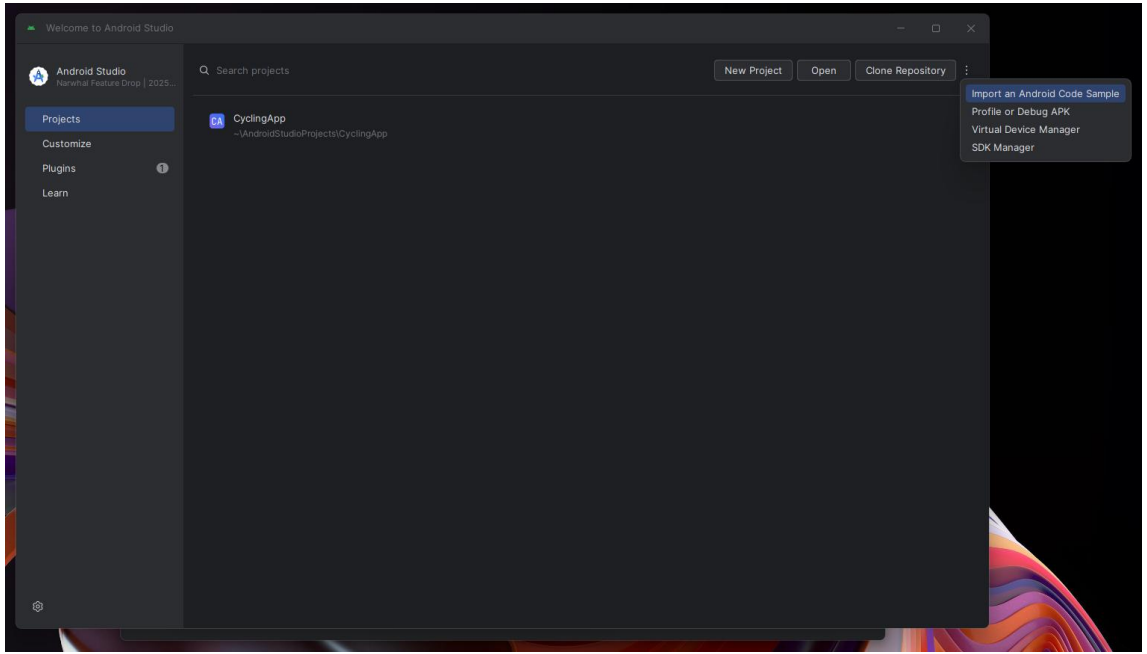


Nota. Ejemplo instalación Android Studio

2. Herramientas necesarias

Android Studio

Imagen 8
Interfaz Android Studio



Nota. Interfaz gráfica de Android Studio, fuente propia

2. Herramientas necesarias
Emulador/dispositivo real

- Para ejecutar nuestras aplicaciones podemos utilizar un emulador de Android
    - En la pantalla principal del SDK, darle a “More Actions” > “Virtual Device Manager” > “Create Virtual Device”.

Imagen 9
Virtual Device Manager

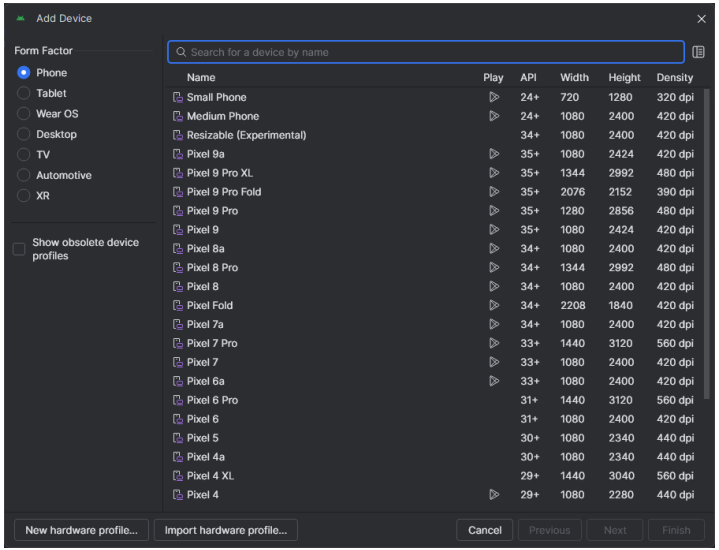


Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

2. Herramientas necesarias

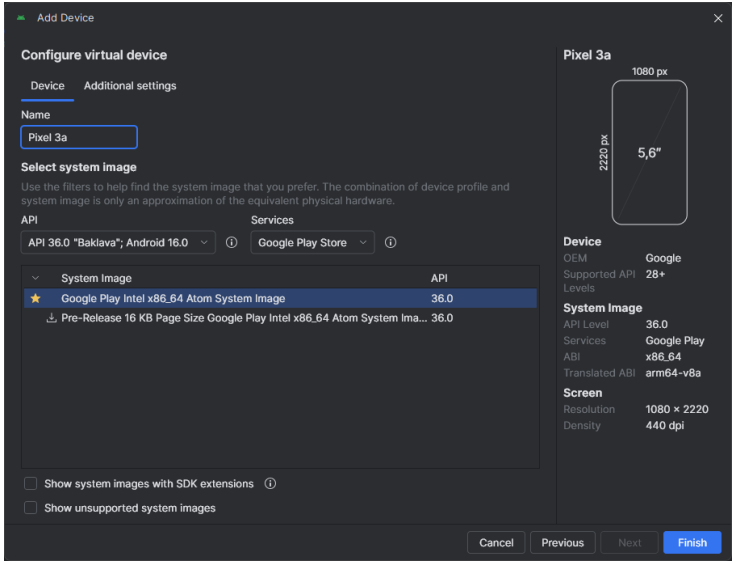
Emulador/dispositivo real

Imagen 9
Tipo de dispositivo



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

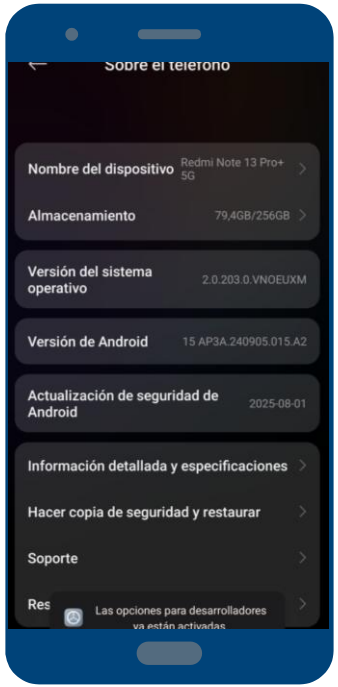
Imagen 10
Características del dispositivo



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

2. Herramientas necesarias
*Emulador/**dispositivo real***

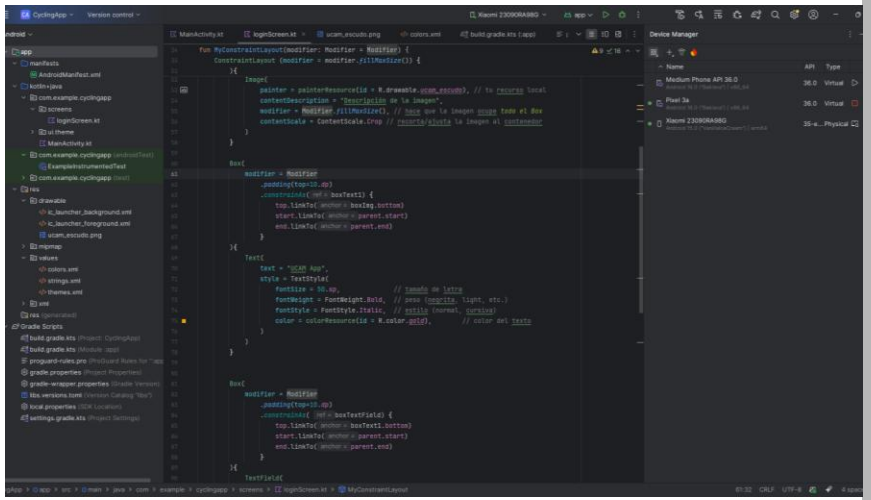
- Ve a Configuración > Acerca del teléfono y pulsa 7 veces en Número de compilación para activar el modo desarrollador.
- Luego, entra en Opciones de desarrollador y activa:
  - Depuración por USB
  - (Opcional) Depuración inalámbrica, si quieres ejecutar sin cable.



2. Herramientas necesarias
*Emulador/**dispositivo real***

- Conectar tu móvil al PC
  - Conecta tu teléfono mediante un cable USB.
  - Es posible que el móvil te pida permitir la depuración USB → acepta.
- Configurar Android Studio
  - Abre tu proyecto en Android Studio.
  - En la barra superior, donde aparece la lista de dispositivos, selecciona tu móvil (debería detectarse automáticamente).
- Haz clic en Run (o Shift + F10).

Imagen 11
Dispositivo real conectado con Android Studio



Nota. Intefaz gráfica de Android Studio con dispositivo real conectado

2. Herramientas necesarias
Gradle

- Gradle es una herramienta de automatización de compilación (build system)
- Compila tu código, gestiona dependencias, empaqueta la app y genera el APK
- Cada proyecto Android tiene al menos dos archivos build.gradle:
    - Nivel de proyecto: configuración general (plugins, repositorios, versión de Gradle).
    - Nivel de módulo (app): configuración de la aplicación (SDK, dependencias, versiones, etc.).

Imagen 12
Icono de gradle



Nota. Icono de gradle

Imagen 1
Título asignado a la imagen

3. Kotlin en Android

Definición



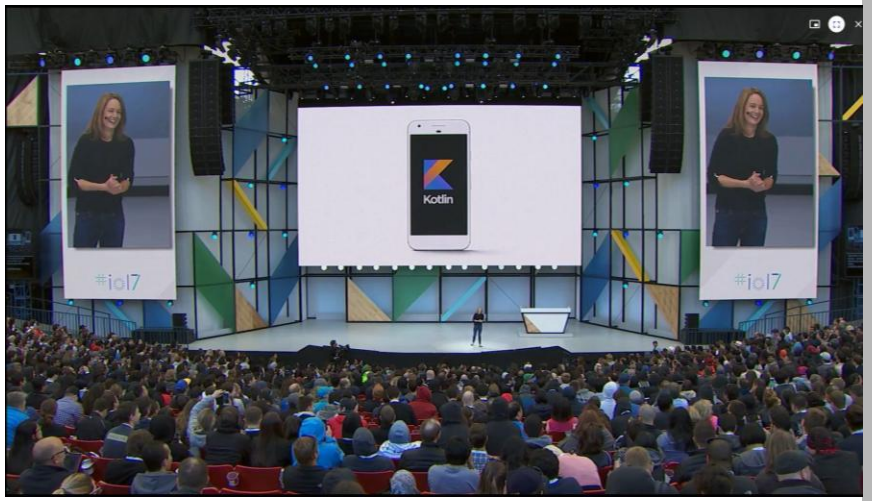
Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

3. Kotlin en Android

¿Por qué Kotlin es el lenguaje oficial?

- Declarado oficial en **2017** por Google
- Moderno y diseñado para mejorar la productividad.
- Beneficios para desarrolladores
  - Menos líneas de código → menos errores.
  - Compatible con todas las herramientas de Android Studio.
  - Comunidad activa y en crecimiento.

Imagen 13
Google IO 2017

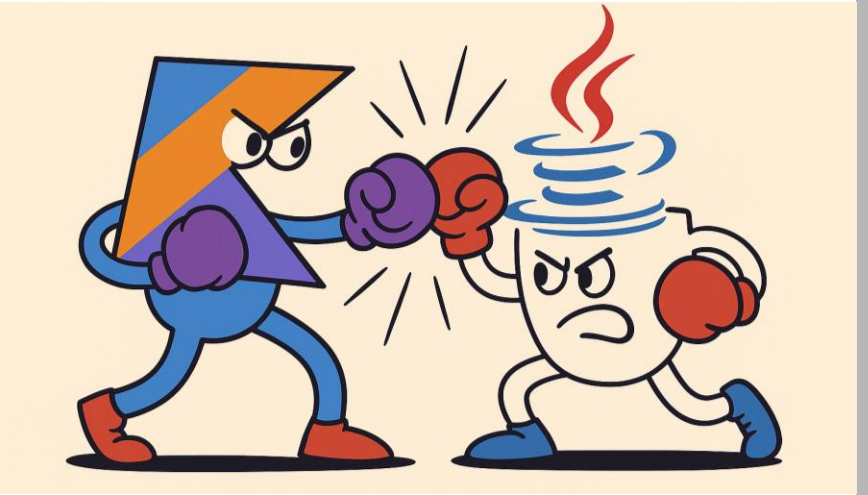


Nota. Foto del evento donde se declaro Kotlin como lenguaje oficial para programar en Android

3. Kotlin en Android
Diferencias con Java

- Código más conciso y legible.
- Manejo seguro de nulls (evita errores comunes).
- Total interoperabilidad con Java (puedes usar ambos juntos).
- Soporte para programación funcional y corutinas (manejo sencillo de procesos asíncronos).

Imagen 14
Kotlin vs Java



Nota. Caricatura de Kotlin luchando contra caricatura de Java

3. Kotlin en Android

Conceptos esenciales:

- **Variables** y **constantes**
  - var: variable mutable.
  - val: variable inmutable (similar a final en Java).
- **Tipos de datos**
  - Números (Int, Double, Float, etc.), Boolean, Char, String.
  - Todo en Kotlin es un objeto (no hay primitivos puros como en Java).
- **Null Safety** (seguridad contra nulls)
  - Evita el clásico NullPointerException.
  - Se usan tipos seguros (String) o anulables (String?).

3. Kotlin en Android

Conceptos esenciales:

- **Funciones**
  - Se definen con fun.
  - Pueden tener valores por defecto y parámetros nombrados.
- **Clases** y **objetos**
  - Soporta POO: clases, objetos, herencia, interfaces.
  - También admite data classes para modelos de datos.
- **Programación funcional**
  - Soporta lambda expressions y funciones de orden superior.
  - Colecciones (listOf, mapOf, setOf) con métodos como map, filter, reduce.

3. Kotlin en Android

Conceptos esenciales:

- **Coroutines**
  - Mecanismo ligero para manejar tareas asíncronas y concurrencia.
- **Extensiones**
  - Extension functions: permiten agregar funciones a clases existentes sin herencia.
- **Interoperabilidad con Java**
  - Kotlin y Java pueden convivir en el mismo proyecto.
  - Permite usar librerías de Java directamente

4. Estructura proyecto Android

Definición

Imagen 15
Arbol tecnológico

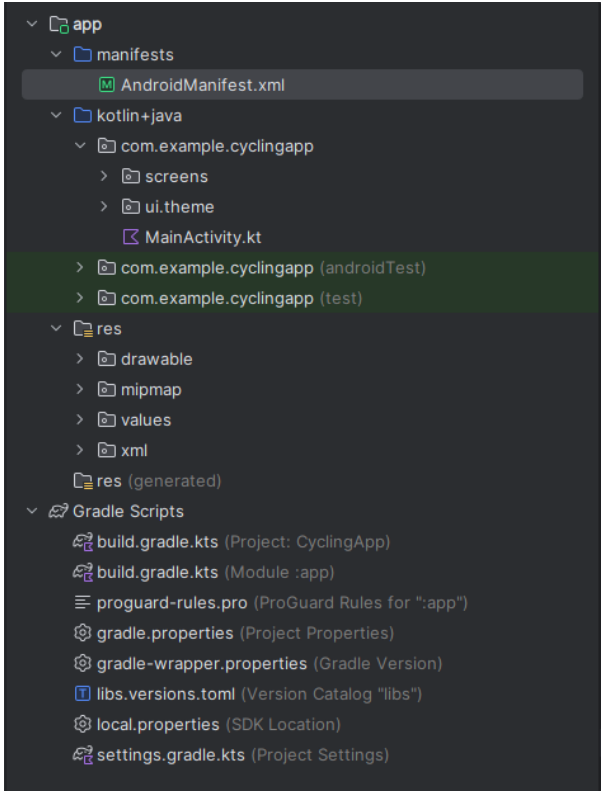


Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

4. Estructura proyecto en Android

Estructura de carpetas

- app/ = corazón del proyecto
  - manifests/ = reglas de la app
  - kotlin+java = clases y lógica de la app
  - res/ = apariencia de la app
- Gradle = compilación y librerías.



4. Estructura proyecto en Android
Manifest

- Datos generales de la app
  - Nombre, icono, tema principal... versionName).
  - Actividades (Activities)
  - Permisos: indican qué recursos del dispositivo necesita la app.
  - Compatibilidad y configuración
    - Versión mínima de Android requerida (minSdkVersion).
    - Orientaciones soportadas (portrait,landscape).
    - Declaración de hardware necesario (ej. cámara, Bluetooth).

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="CyclingApp"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.CyclingApp">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="CyclingApp"
            android:theme="@style/Theme.CyclingApp">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

4. Estructura proyecto en Android

Kotlin+java

```
com.example.myapp
├─ ui/                    → Todo lo relacionado con la interfaz
│   ├─ theme/             → Colores, tipografía, estilos (MaterialTheme)
│   ├─ components/        → Composables reutilizables (Botones, Cards, etc.)
│   ├─ screens/           → Pantallas principales (HomeScreen, LoginScreen, etc.)
│   └─ navigation/        → Manejo de navegación con NavHost y rutas
│
├─ data/                  → Capa de datos
│   ├─ model/             → Clases de datos (User, Product, etc.)
│   ├─ repository/        → Acceso a datos (APIs, BD local, etc.)
│   └─ network/           → Configuración de Retrofit, servicios REST
│
├─ domain/                → (Opcional) Reglas de negocio
│   └─ usecases/          → Casos de uso (GetUserUseCase, SaveProductUseCase)
│
├─ di/                    → Inyección de dependencias (Hilt, Koin, etc.)
│
├─ utils/                 → Funciones auxiliares (extensiones, validaciones)
│
├─ MainActivity.kt         → Punto de entrada de la app
└─ MyApp.kt                → Setup de la app con Compose (MaterialTheme, NavHost)
```

4. Estructura proyecto en Android

res/

- **drawable**/
  - Imágenes en formato PNG, JPG, SVG (vector drawables).
  - Íconos que quieras usar en tus Composables
- **mipmap**/: Iconos de la aplicación en distintas resoluciones (para el launcher).
- **values**/ archivos XML con recursos reutilizables:
  - colors.xml
  - strings.xml
  - themes.xml → usado si mezclas Compose con Views o Material Components clásicos.
- **raw**/: archivos de datos en crudo, como audios, JSON o textos que la app necesita.

4. Estructura proyecto en Android

Gradle. Añadir dependencias

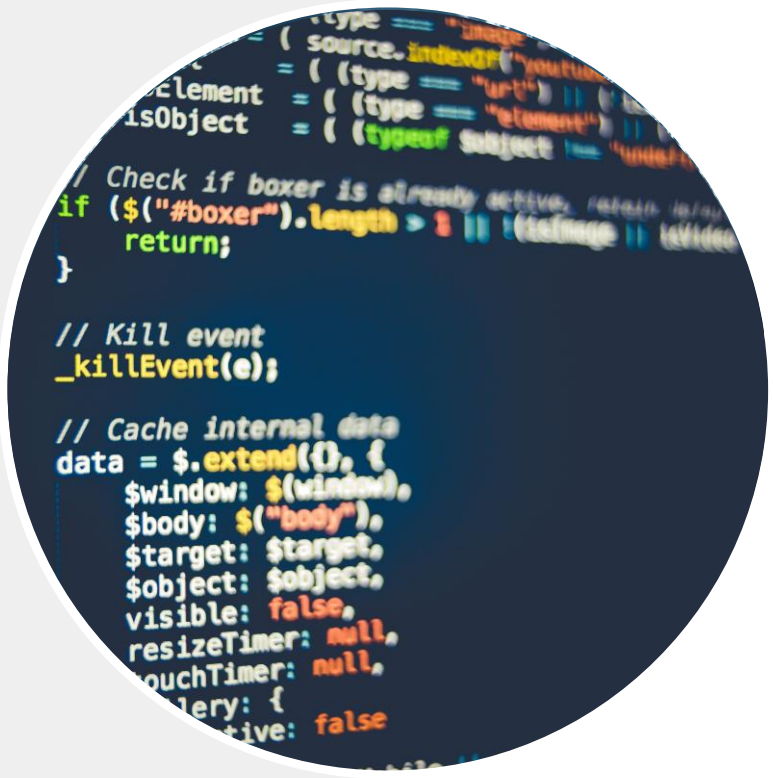
- Abres app/build.gradle (o build.gradle.kts).
- Dentro de dependencies { }, añades la librería con implementation "grupo:libreria:versión".
- Haces Sync Project with Gradle Files en Android Studio (botón arriba o automáticamente).



5. Componentes de una app de Android Compose

Definición

Imagen 15
Arbol tecnológico



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

5. Componentes de una app de Android Compose
Activities

- Anteriormente una app de Android contaba con Activities, Fragments, Views, etc.
- Cada activity eran las pantallas de la aplicación
- Actualmente solo cuenta con un **MainActivity**
- El paso entre pantallas se consigue con navegación interna

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MyApp() // Aquí empieza Compose
        }
    }
}
```

5. Componentes de una app de Android Compose
Composables

- Funciones anotadas con **@Composable.**
- Son los bloques de construcción de la UI.
- Ejemplos básicos: Text, Button, Image, Column, Row

```
@Composable
fun Greeting(name: String) {
    Text("Hola $name!")
}
```

5. Componentes de una app de Android Compose
Estados (states)

- La UI en Compose depende del estado
- Se maneja con
  - Remember
  - mutableStateOf
  - ViewModel
  - StateFlow

```
var count by remember { mutableStateOf(0) }
Button(onClick = { count++ }) {
    Text("Clicks: $count")
}
```

5. Componentes de una app de Android Compose

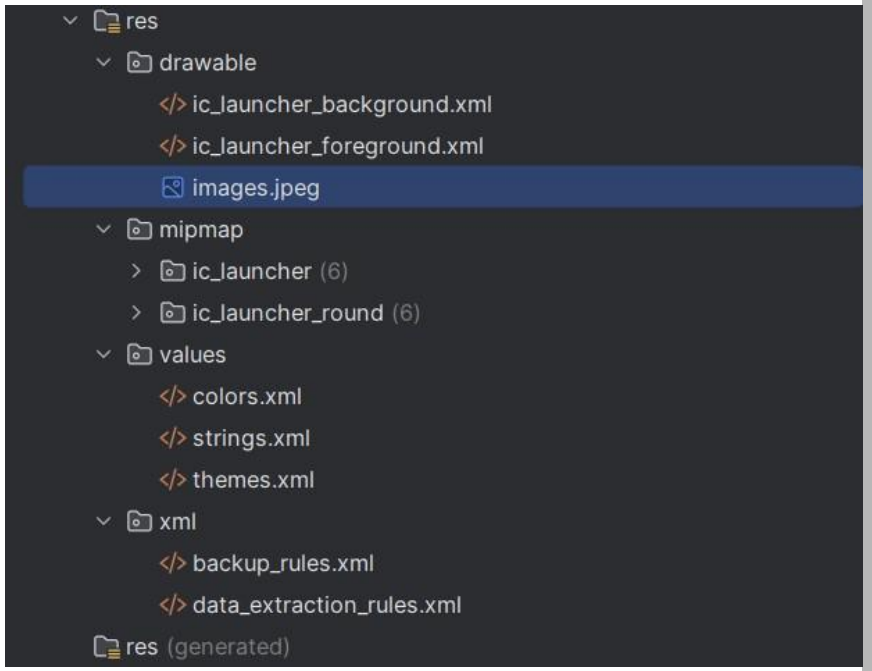
Navigation Component

- Maneja la transición entre pantallas dentro de una sola Activity.
- Con Compose se usa
  - NavHost
  - NavController.

```
@Composable
fun NavigationWrapper() {
   val navController = rememberNavController() //Indica a que
componente de navegación se va a dirigir
   NavHost(navController = navController, startDestination = Login) {
      //Aquí van las rutas de navegación
      composable<Home> {
         HomeScreen({navController.navigate(Login)})
      }
      composable<Login> {
         LoginScreen({navController.navigate(Home)})
      }
   }
}
```

5. Componentes de una app de Android Compose
Recursos

- Aunque la UI se hace en Kotlin, seguimos teniendo:
  - res/values/strings.xml → textos.
  - res/drawable/ → imágenes.
  - res/colors.xml y themes.xml → colores/estilos.
- Estos recursos se integran en Compose con stringResource(), painterResource(), etc.



5. Componentes de una app de Android Compose

Persistencia y APIs

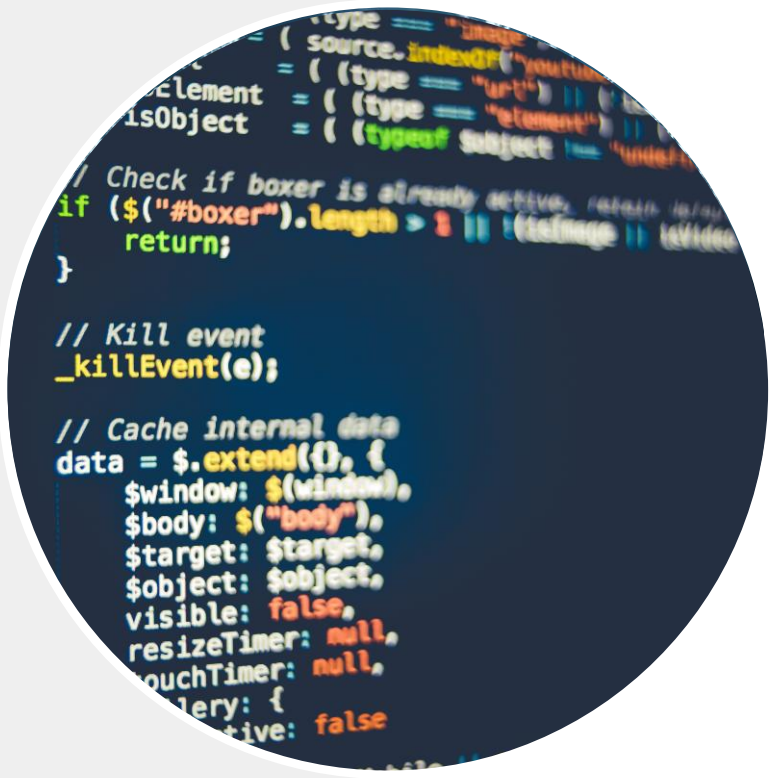
- Room Database para datos locales.
- Retrofit + coroutines para conectarse a APIs.
- Los resultados se entregan al ViewModel → Compose renderiza en base al estado.



6. Copilot en Android Studio

Definición

Imagen 15
Arbol tecnológico



Nota. Adaptado de Título de la imagen, por Autor, Año, Web (con hipervínculo)

6. Copilot en Android Studio

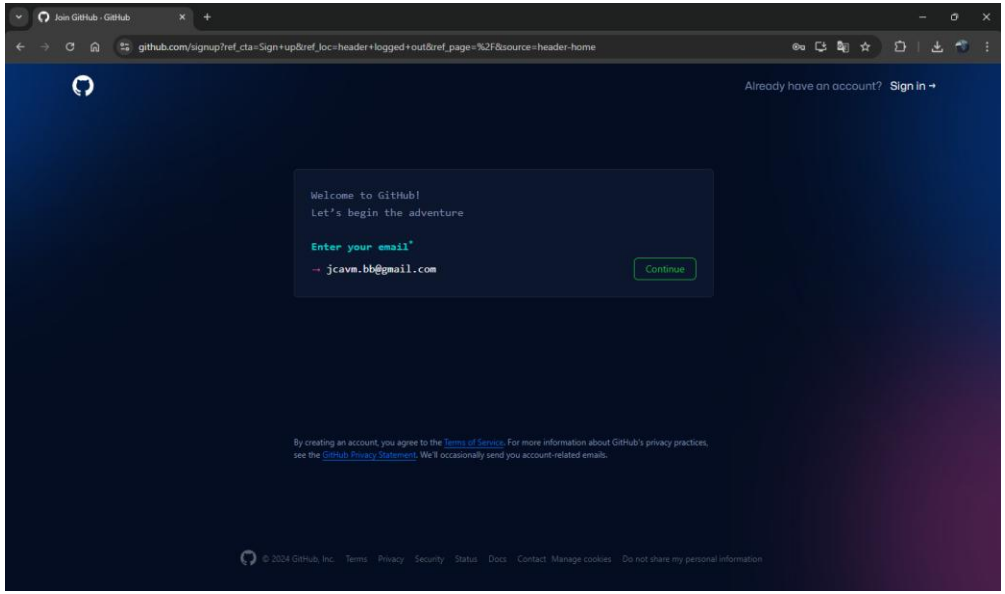
Introducción

- ✓ Es una plataforma de desarrollo colaborativo para aplicaciones de software
- ✓ Utiliza el sistema de control de versiones de Git
- ✓ Ventajas de usar GitHub
  - Control de versiones
  - Colaboración y trabajo en equipo
  - Almacenamiento centralizado
  - Automatización e integración continua
  - Etc.

6. Copilot en Android Studio

Crear cuenta

- ✓ Nos dirigimos a www.github.com
- ✓ Le damos a Sign up
- ✓ Introducimos un email, password, etc.
- ✓ Seleccionar la opción Free



6. Copilot en Android Studio

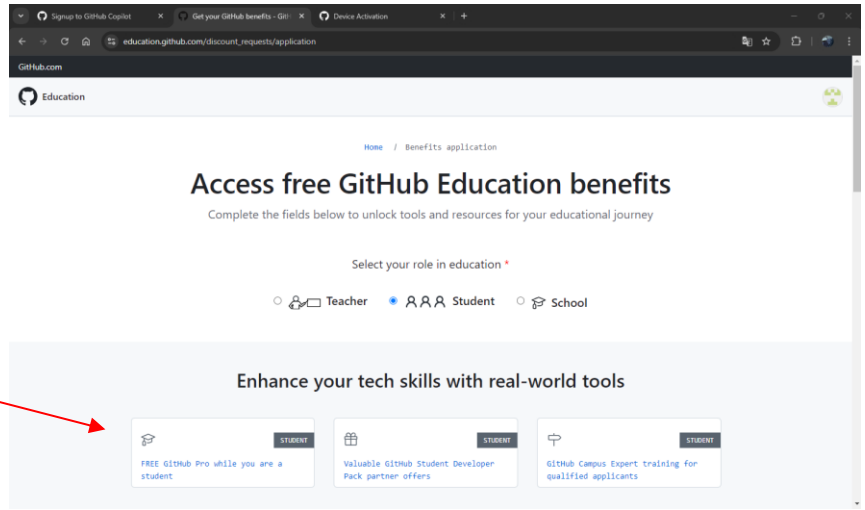
Copilot

- ✓ Es una herramienta de inteligencia artificial basada en la nube desarrollada por GitHub y OpenAI para ayudar y asistir a los usuarios de IDEs de JetBrains
- ✓ Ayuda a crear código de manera automática
- ✓ Tenéis licencia gratuita desde la universidad

6. Copilot en Android Studio

Copilot: GitHub Education

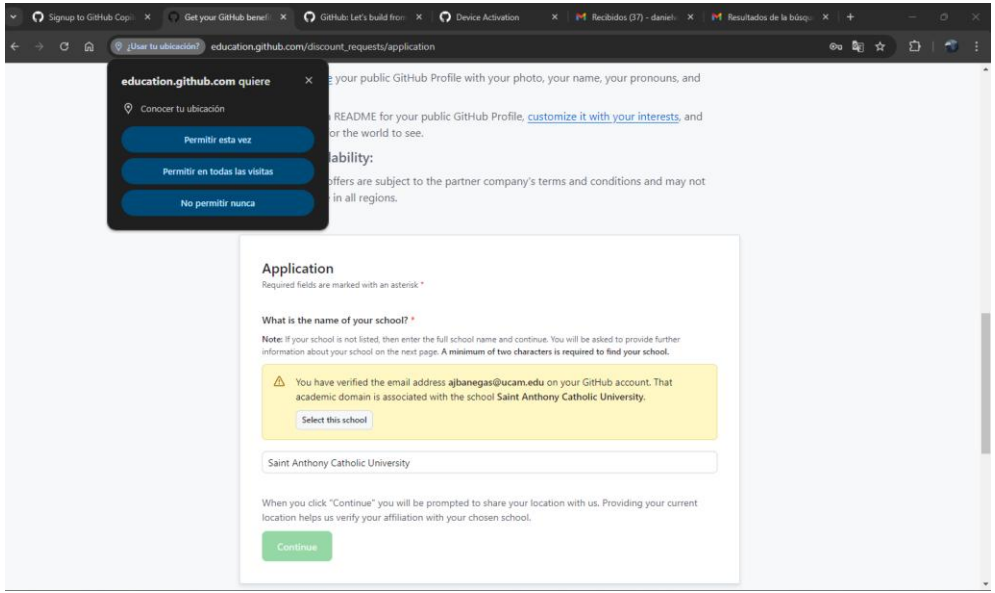
- ✓ 1. Accedemos a https://education.github.com/discount_requests/application
- ✓ 2. Seleccionamos el rol de estudiante



6. Copilot en Android Studio

Copilot: GitHub Education

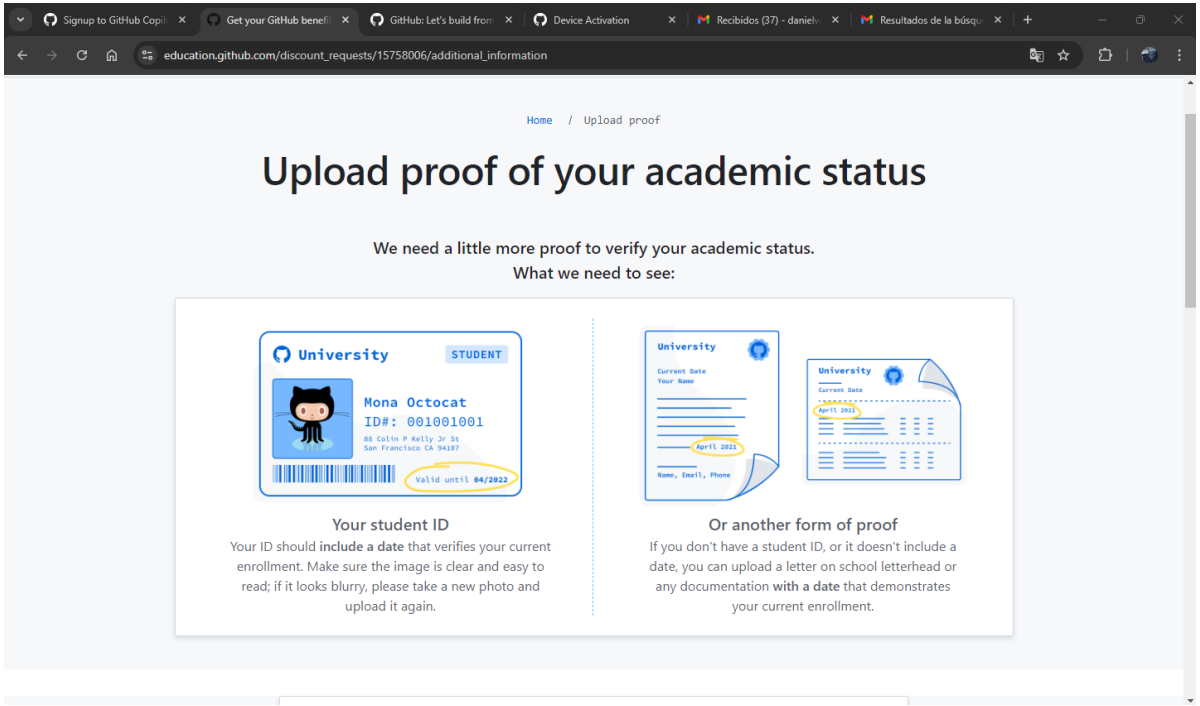
- ✓ 3. En Name of your school: Saint Anthony Catholic University
- ✓ 4. Habilitáis la ubicación



6. Copilot en Android Studio

Copilot: GitHub Education

- ✓ 5. Os pedirá una muestra de identificación



6. Copilot en Android Studio

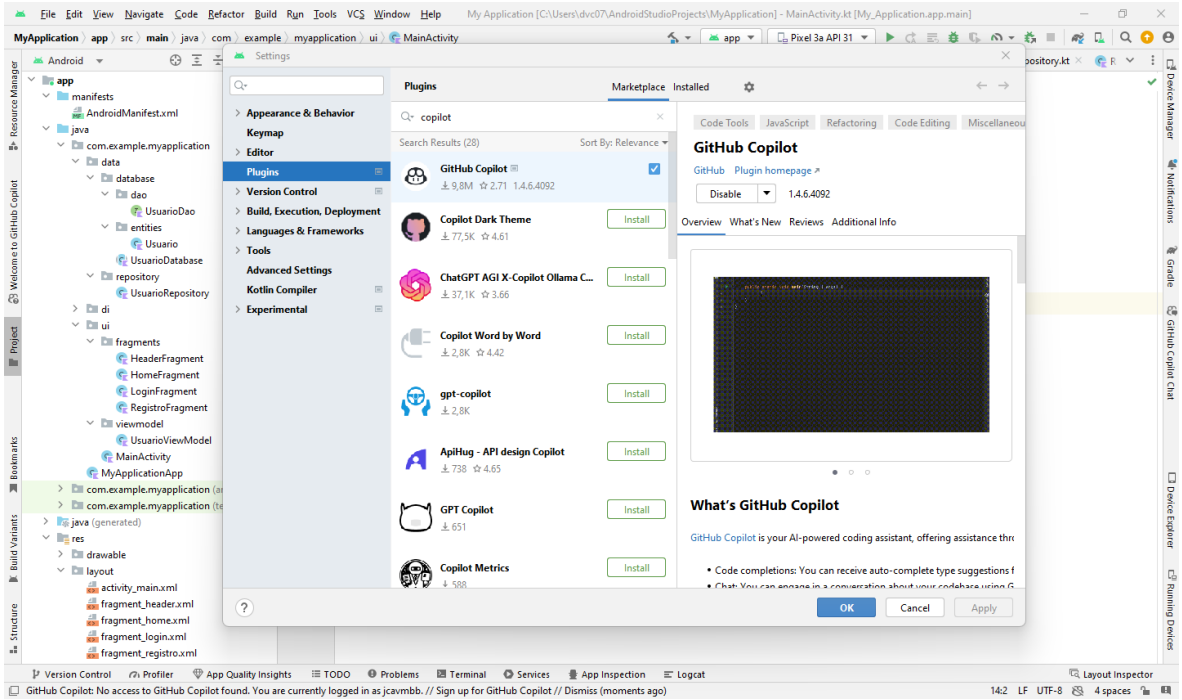
Copilot y Android Studio

- ✓ Una vez que hayáis obtenido el acceso a Copilot podréis sincronizarlo con vuestro IDE de JetBrains
- ✓ Para usarlo en Android Studio tenéis que instalar el plugin

6. Copilot en Android Studio

Copilot: instalación plugin

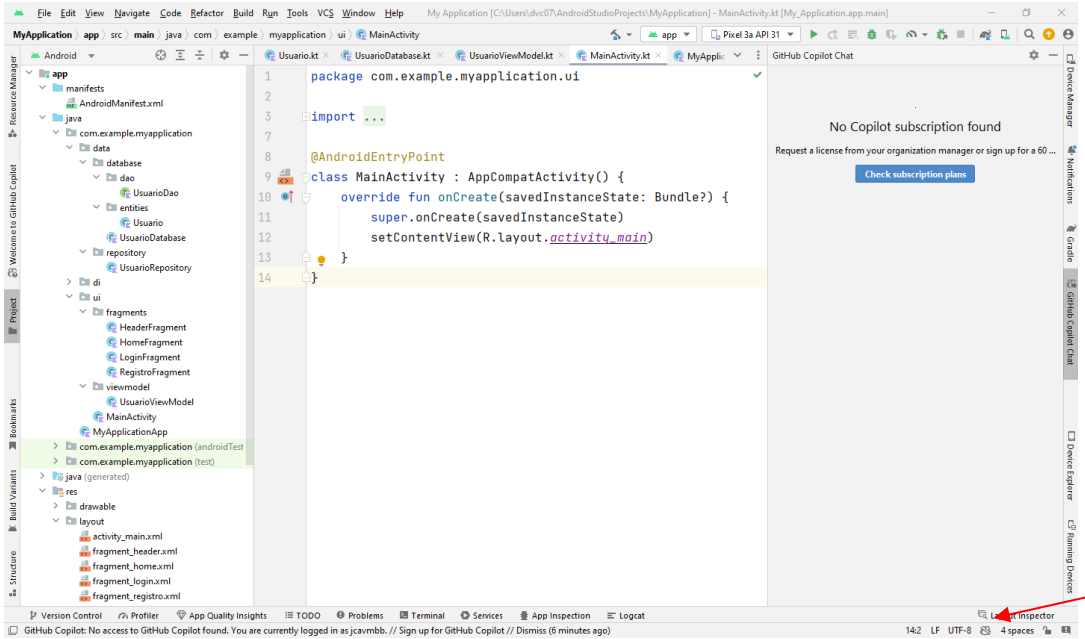
- ✓ Nos vamos a File->Settings->Plugins, buscamos GitHub Copilot y lo instalamos



6. Copilot en Android Studio

Copilot: Plugin

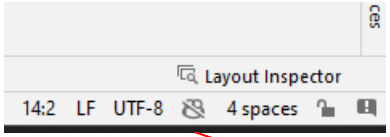
- ✓ En la parte inferior y derecha os aparecerá la parte de GitHub Copilot

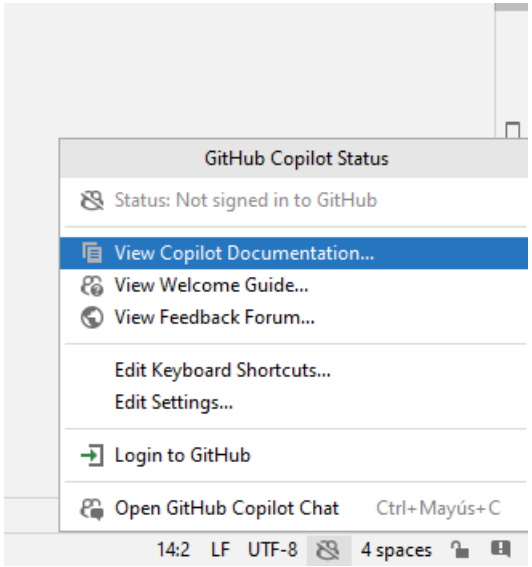


6. Copilot en Android Studio

Copilot: Plugin

- ✓ Iniciar Copilot clicando sobre el botón



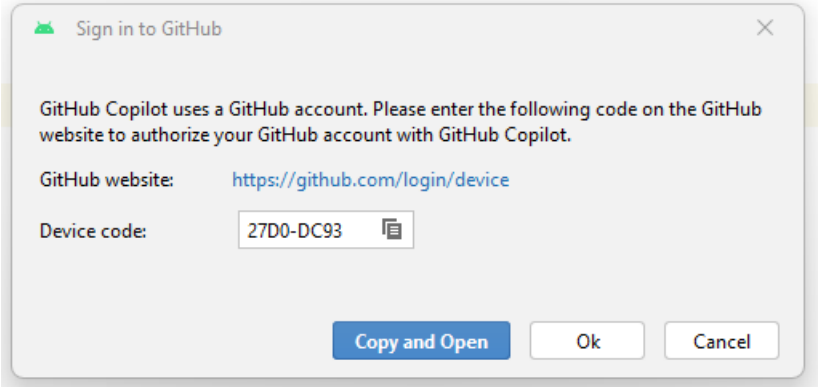


- ✓ Le damos a Login to GitHub

6. Copilot en Android Studio

Copilot: Plugin

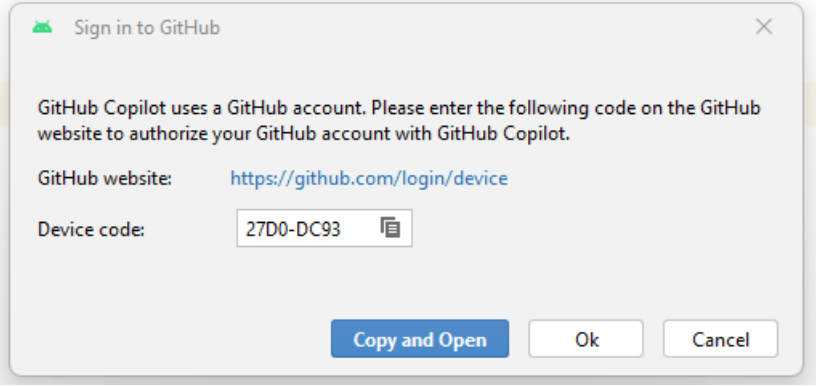
- ✓ Nos dará un código que meteremos en la web

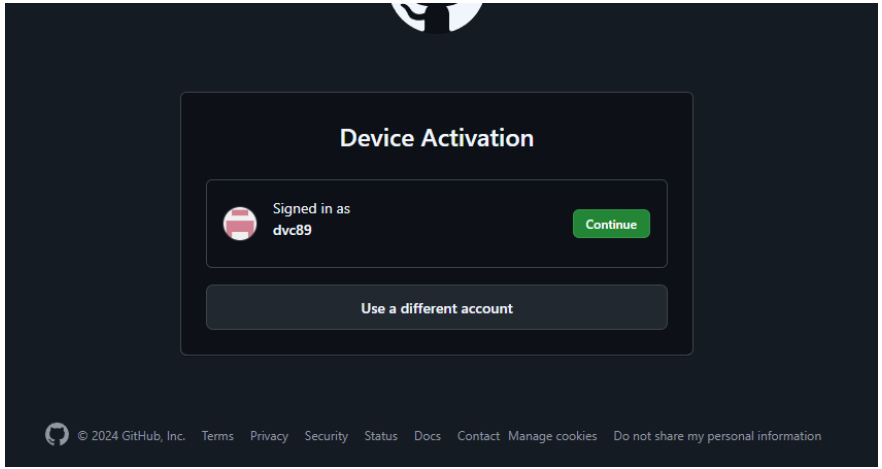


6. Copilot en Android Studio

Copilot: Plugin

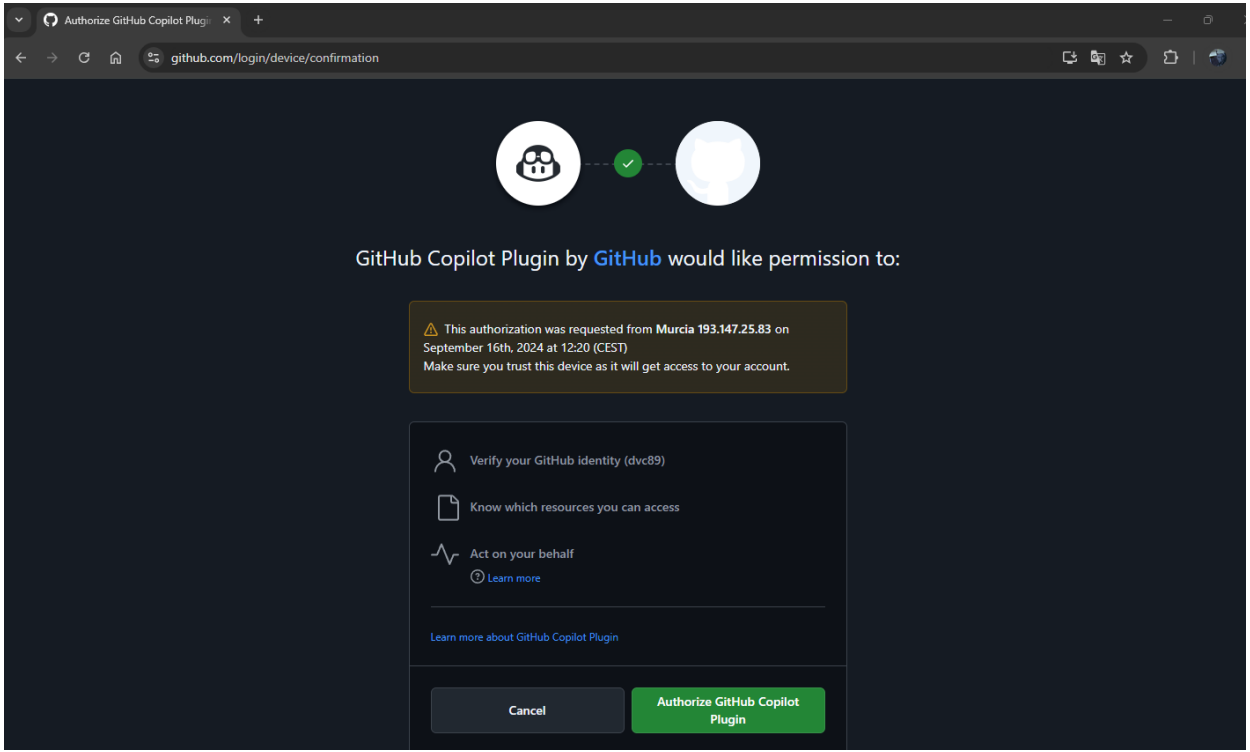
- ✓ Nos dará un código que meteremos en la web





6. Copilot en Android Studio

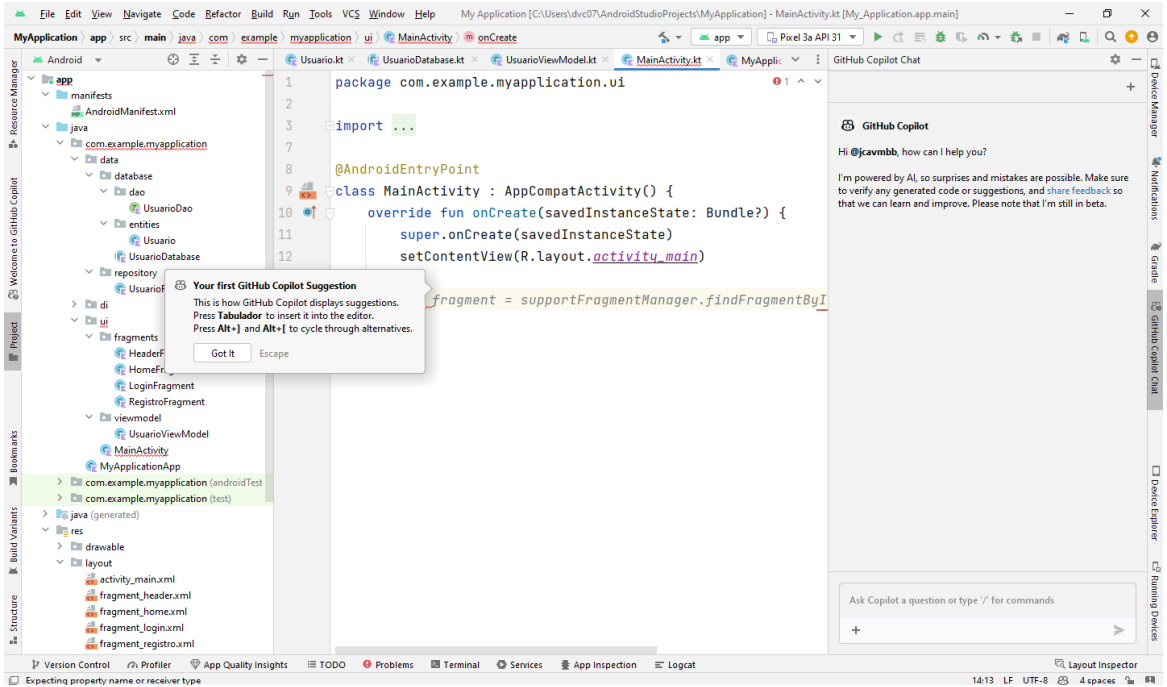
Copilot: Plugin



6. Copilot en Android Studio

Copilot: Plugin

- ✓ Si todo ha ido bien os pondrá un mensaje de éxito y podréis utiliza Copilot



REFERENCIAS

BIBLIOGRÁFICAS

Android Studio: https://developer.android.com/

Jetpack Compose: https://developer.android.com/compose

OpenCompose: https://www.jetpackcompose.pro/

Daniel Valero Carreras

dvalero@ucam.edu

UCAM Universidad Católica de Murcia



© UCAM